

Artificial Neural Network

In this Assignment I need to Construct the Neural Network from scratch without using Any inbuilt functions of pytorch.

Dataset

MNIST Hand written digits that data is converted to 0 to 1 Scale

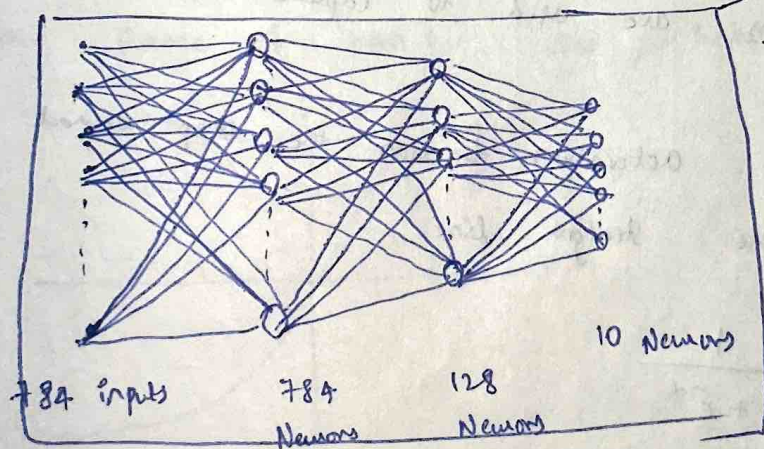
Train Data, Test data from Torchvision Datasets I'm utilizing the mnist Dataset

Architecture.

→ Here we are using 1 input layer 1 hidden layer and 1 output layer

→ So the Architecture look like is

This Architecture is wrong there are 1 hidden 1 output layer only



→ Number of trainable parameters

$$784 \times 128 + 128 \\ = 785 \times 128 \\ +$$

$$128 \times 10 + 10 \\ = 129 \times 10$$

$$= 785 \times 128 + 1290 = 101770$$

The weights and bias values I used with HE Initialization and the Xavier Initialization

Xavier Initialization

Weights of the Neural Network we set as the

$$W \sim N\left(0, \frac{2}{fan_{in} + fan_{out}}\right)$$

But in our code I used as $N\left(\frac{1}{fan_{in}}\right)$

He initialization

The weights of the neural network set as the

$$w \sim \mathcal{N}\left(0, \frac{2}{fanin}\right)$$

Here in my code also I used $w \sim \mathcal{N}\left(0, \frac{2}{fanin}\right)$

Here $fanin = 784$

Activation functions

I used 3 different activation function to compare how the neural network is performing in different activation

to the activation functions that are used are

1. Sigmoid

2. ReLU

3. tanh

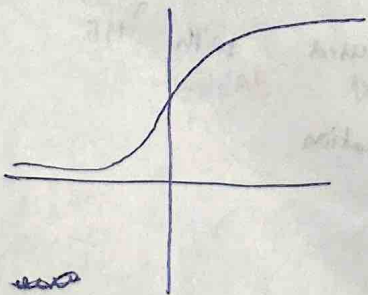
These activation functions are used to capture the nonlinearity in the data

If I don't use the activation function the deep neural network behave as the straight line.

Sigmoid

÷

$$\frac{1}{1 + e^{-x}}$$



The Range of this activation lies between 0 to 1

* When x become $+\infty$ then Sigmoid become 1

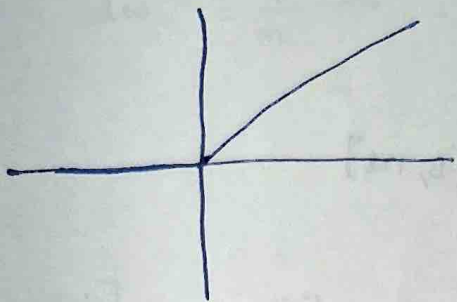
* When x become $-\infty$ then Sigmoid become 0

→ In Back-propagation the derivative of the Sigmoid

$$\sigma(x) (1 - \sigma(x))$$

ReLU

$$f(x) = \begin{cases} x & x > 0 \\ 0 & x \leq 0 \end{cases}$$



The Range of this activation from 0 to ∞

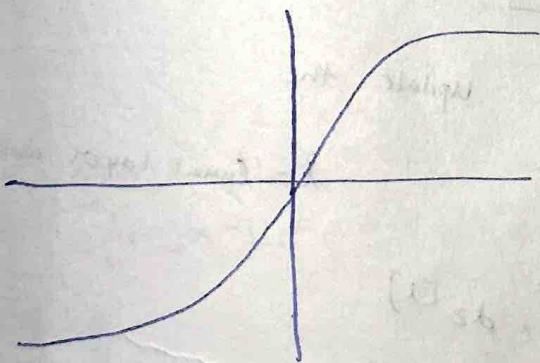
In the Backpropagation the Derivative of ReLU

$$f'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$$

tanh

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

the Range of tanh lies between $[-1, 1]$



In Backpropagation the Derivative of tanh is

$$f'(x) = 1 - \tanh^2(x)$$

Loss function

The Output layer is containing multi class classification so we can use the Categorical cross entropy

$$Loss = \sum_{i=1}^n -y_i \log \hat{y}_i$$

Forward propagation

- From the data loader $[B, 784]$ 2 get so this is passed to the NN that contain $[784, 128]$

- So the shape become

$$[B, 784] \cdot [784, 128] = [B, 128]$$

- $[B, 128]$ pass to weights $[128, 10]$

$$[B, 128] \cdot [128, 10] = [B, 10]$$

- Now Categorical Loss is Applied then Back propagation is done

Back propagation

- Weights are updated via the Gradient descent with Back propagation

- Formula of Gradient descent

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

- The main important formula to update the

$$dA^{[L-1]} = dz^{[L]} \cdot (w^{[L]})^T$$

L = Current layer number

$$dw^{[L]} = \frac{1}{m} (A^{[L-1]})^T \cdot dz^{[L]}$$

$$db^{[L]} = \frac{1}{m} \sum dz^{[L]}$$

- So In our Architecture

$$1) dz^{[2]} \text{ shape } [B, 10]$$

$$2) dw^{[2]} = \frac{1}{m} (A^{[1]})^T \cdot dz^{[2]} = [128, B] [B, 10] = [128, 10]$$

So the weights between Hidden layer and output layer shape

$$3) db^{[2]} = [128, 10] \text{ Sum along axis } = 0 \text{ so } [1, 10]$$

$$4) dA^{[1]} = dz^{[2]} \cdot (w^{[2]})^T = [B, 10] \cdot [10, 128] = [B, 128]$$

$$5) dz^{[1]} = [B, 128]$$

$$6) dw^{[1]} = \frac{1}{m} (A^{[0]})^T \cdot dz^{[1]} = [784, B] \cdot [B, 128] = [784, 128]$$

$$7) db^{[1]} = [1, 128]$$